

Sistemas Distribuídos

8ª aula

Message Passing Interface

Prof. DSc. Douglas Oliveira

douglas.oliveira@uva.br

- MPI define uma biblioteca padrão para passagem de mensagens que pode ser usada no desenvolvimento de programas portáveis usando C ou Fortran.
- O padrão MPI define tanto a sintaxe quanto a semântica de um conjunto básico de rotinas.
- Implementações de MPI estão disponíveis em quase todos os computadores paralelos comerciais.
- É possível escrever programas com passagem de mensagens plenamente funcionais usando apenas as seis rotinas mostradas a seguir.

- MPI_Init é chamada antes de qualquer outra rotina MPI. Seu propósito é inicializar o ambiente MPI.
- MPI_Finalize é chamada no final da computação, e ele executa várias tarefas de limpeza para finalizar o ambiente MPI.
- Os protótipos destas duas funções são:

```
int MPI_Init(int *argc, char ***argv)
```

```
int MPI_Finalize()
```

- MPI_Init também extrai qualquer argumento relacionado ao MPI da linha de comando.
- Todas rotinas MPI, tipos de dados, e constantes tem o prefixo “MPI_”. O código de retorno quando a operação é completada com sucesso é MPI_SUCCESS.

- As funções `MPI_Comm_size` e `MPI_Comm_rank` são usadas para determinar o número de processos e o rótulo do processo chamador, respectivamente.
- As chamadas destas rotinas são as seguintes:
`int MPI_Comm_size(MPI_Comm comm, int *size)`
`int MPI_Comm_rank(MPI_Comm comm, int *rank)`
- O ranque de um processo é um inteiro que varia de zero até o tamanho do comunicador menos um.

- As funções básicas para enviar e receber mensagens em MPI são o MPI_Send e MPI_Recv, respectivamente.
- As chamadas destas funções são as seguintes:

```
int MPI_Send(void *buf, int count, MPI_Datatype  
datatype, int dest, int tag, MPI_Comm comm)
```



```
int MPI_Recv(void *buf, int count, MPI_Datatype  
datatype, int source, int tag, MPI_Comm comm,  
MPI_Status *status)
```
- MPI oferece tipos de dados equivalentes a todos os tipos de dados de C. Isto é feito devido a questões de portabilidade.